Introduction to agentic AI

TECH2: Introduction to Programming, Data, and Information Technology

Richard Foltyn

NHH Norwegian School of Economics

November 4, 2026

Before we begin ...

1. **Course evaluation**
   - Reminder: fill out the TECH2 course evaluation
2. **Schoolyear exam environment tryout**
   - **Thursday, November 12, 10:15–11:00**, in **Aud B**
   - Test logging into Schoolyear and get familiar with the environment
   - Check the keyboard settings, particularly on Macs and especially with non-Norwegian or non-US keyboards

Contents

1 The AI industry

2 Working effectively with AI agents

3 Customizing your agent

4 Hands-on session

5 Cheat sheets for CLI agents

6 Additional resources

The AI industry

Your turn

Which company do you think is the most important in the AI industry?

The biggest AI industry players (w/o China)

Frontier labs: leading AI models

Company	USD bn
Google (Alphabet)	3,440
SpaceX (xAI / Grok)	1,830
OpenAI	?
Anthropic	?

AI chips: design and manufacturing

Company	USD bn
Nvidia	4,940
AMD	824
TSMC (foundry)	2,106
Broadcom	1,560

Hyperscalers: large data centers

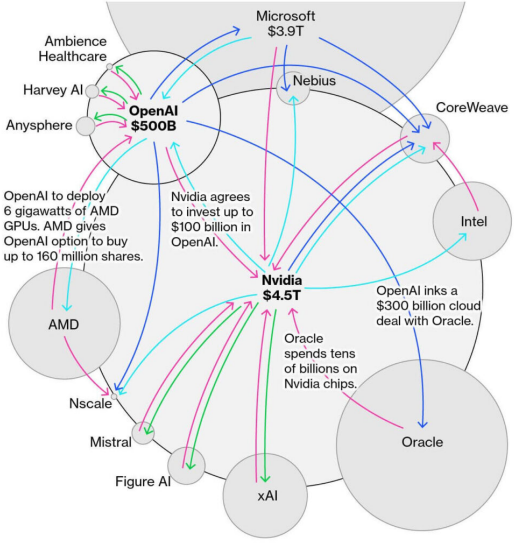
Company	USD bn
Microsoft	3,880
Google (Alphabet)	3,440
Amazon	2,600
Meta	1,570
Oracle	396

Memory chips

Company	USD bn
Samsung Electronics	1,077
Micron	1,000
SK hynix	775

Valuations as of Oct 30, 2026. For comparison: Norway's oil fund ≈ USD 2,293 bn (Sept 30, 2026).

The AI money machine: October 2025



Relationships

- Hardware / software
- Investment
- Services
- Venture capital

Circles sized by market value.

Source: Bloomberg News, October 2025.

AI providers: models and tools

	Anthropic	OpenAI	Google	SpaceXSI
Premium reasoning	Fable 5.1	GPT-6 Astra	—	—
Main models	Opus 5.5	GPT-6.1 Sol	Gemini 4 Argon	Grok 4.7
Fast / lower cost	Sonnet 5.5 Haiku 5.5	GPT-6.1 Terra GPT-6 Luna	Gemini 3.8 Flash	—
Desktop work tools	Claude Cowork	ChatGPT Work	—	—
Coding apps / editors	Claude Code (VS Code)	Codex (VS Code)	Antigravity (app / editor)	Cursor
CLI tools	Claude Code	Codex	Antigravity CLI	Cursor CLI

Chinese AI labs: open-weight model families

	DeepSeek	Z.ai	Moonshot AI	Alibaba	MiniMax
Main models	DeepSeek V4 Pro	GLM-5.3	Kimi K3.1 Pro	Qwen3.8 Max	M3
Fast / lower cost	DeepSeek V4.1 Flash	GLM-5.3 Flash	—	Qwen3.8 Flash-Next	M3.1 Flash

Third-party, model-agnostic harnesses

	 **OpenCode**	 **pi**	 **Cline**
Main strength	Coding agent with multiple interfaces	Minimal core; highly customizable	Visual workflow; plan, act, undo
Desktop app	Yes	—	Yes (beta)
VS Code extension	—	—	Yes
Terminal (CLI)	`opencode`	`pi`	`cline`
Anthropic integration	API only	Subscription via Claude Code plugin or API	Subscription via Claude Code or API
OpenAI integration	Subscription or API	Subscription or API	Subscription or API
Google integration	Subscription via plugin or API	Subscription via plugin or API	API only

Your turn

Explain what a token is in the context of a large language model (LLM).

Billing methods for AI access

Subscriptions

- Individual plans can be heavily subsidized versus API prices
- Enterprises generally use separate plans. Pricing and usage terms differ
- A subscription does not automatically cover separate API usage

Direct lab APIs

- API key from the lab, payment directly to the lab
- Usage-based billing
- Token rates depend on model complexity

Free tier

- No-cost access with usage limits

API aggregators / AI gateways

- Example: OpenRouter
- Access to many models and providers through one service
- No need for lab-specific API keys

Curated model services

- Example: OpenCode Zen / Console
- Prepaid credits for selected models, billed by usage

Individual plans (USD per month)

	Anthropic	OpenAI	Google	SpaceXSI
Free	Chat / apps	Chat / apps	Antigravity CLI Limited usage	Cursor CLI Limited usage
Basic	—	Go ($8) Chat / apps	AI Plus ($7.99) Antigravity CLI	Plus ($8) Cursor CLI
Around $20	Pro ($20) Claude Code	Plus ($20) Codex CLI	AI Pro ($20) Antigravity CLI	Premium ($30) Cursor CLI
$100	Max 5x Claude Code	Pro Codex CLI	AI Ultra 5x Antigravity CLI	Super Cursor CLI
$200	Max 20x Claude Code	Pro Codex CLI	AI Ultra 20x Antigravity CLI	Ultra Cursor CLI

Your turn

Look up the input API prices on OpenRouter (https://openrouter.ai) for the following models:

- Claude Fable 5.1
- GPT-6.1 Sol
- Gemini 3.8 Flash
- DeepSeek V4.1 Flash
- GLM 5.3 Flash

Order the models from cheapest to most expensive per million input tokens. Use the lowest standard input price, excluding cache hits, batch pricing, and Flex discounts.

Advertised API prices

Prices in USD per million tokens, ordered by input price (November 4, 2026):

Model	Provider	Input	Output	Cache read
GLM 5.3 Flash	Relace	0.0283 (1.0×)	0.0933	0.0283
DeepSeek V4.1 Flash	Morph	0.035 (1.2×)	0.315	0.006
Gemini 3.8 Flash	Google AI Studio	0.75 (26.5×)	3.75	0.075
GPT-6.1 Sol	OpenAI	2.00 (70.7×)	10.00	0.10
Claude Fable 5.1	Anthropic	10.00 (353.4×)	50.00	0.25

Multiples relative to GLM's input price. Lowest standard input price; all rates in a row use the named provider. Prices change; cache discounts vary by provider.

Cost of solving the term paper

Average per run across three runs per model, ordered by cost:

Model	Thinking	Cost (USD)	Total tokens
DeepSeek V4.1 Flash	max	0.1059 (1.0×)	2,946,128
GLM 5.3 Flash	max	0.1234 (1.2×)	2,756,896
GPT-6.1 Sol	xhigh	0.4721 (4.5×)	885,857
Gemini 3.8 Flash	high	0.7100 (6.7×)	2,872,406
Claude Fable 5.1	xhigh	4.0851 (38.6×)	1,455,991

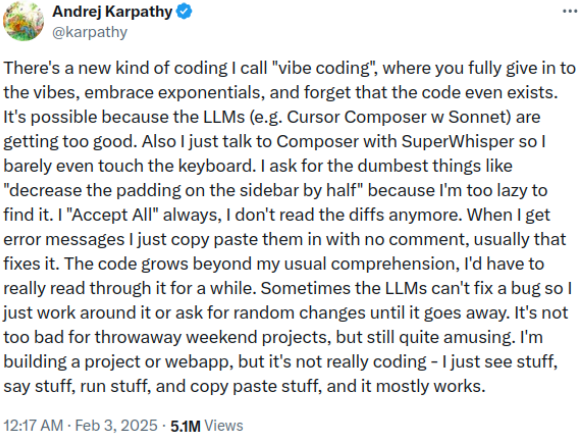
Cost multiples relative to DeepSeek.

- DeepSeek and GLM generate about 3× as many output tokens as GPT
- Their price advantage is smaller for cached input

$\Longrightarrow$ A lower token price can still mean a higher total cost.

Working effectively with AI agents

Vibe coding = an LLM writes the code based on nontechnical instructions



Andrej Karpathy ✓
@karpathy

There's a new kind of coding I call "vibe coding", where you fully give in to the vibes, embrace exponentials, and forget that the code even exists. It's possible because the LLMs (e.g. Cursor Composer w Sonnet) are getting too good. Also I just talk to Composer with SuperWhisper so I barely even touch the keyboard. I ask for the dumbest things like "decrease the padding on the sidebar by half" because I'm too lazy to find it. I "Accept All" always, I don't read the diffs anymore. When I get error messages I just copy paste them in with no comment, usually that fixes it. The code grows beyond my usual comprehension, I'd have to really read through it for a while. Sometimes the LLMs can't fix a bug so I just work around it or ask for random changes until it goes away. It's not too bad for throwaway weekend projects, but still quite amusing. I'm building a project or webapp, but it's not really coding - I just see stuff, say stuff, run stuff, and copy paste stuff, and it mostly works.

12:17 AM · Feb 3, 2025 · **5.1M** Views

- Andrej Karpathy: DeepMind → OpenAI → Tesla → Anthropic

Reading the code?



Guillermo Rauch
@rauchg

If you're not reading the code, whether explicitly or through agentic inquiry, one or more of these is true:

- You're a beginner
- Software is throwaway
- You're prototyping
- You have no users / revenue
- You're taking on debt & risk
- Your problems are basic

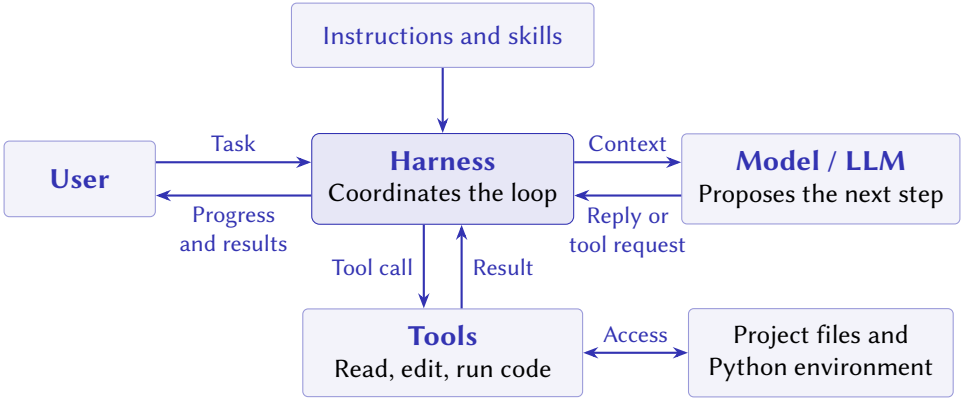
And btw.

6:01 PM · Aug 9, 2026

- Guillermo Rauch: CEO of Vercel, the company behind the Next.js web framework and v0, an AI tool for building web apps from prompts

The agent system: model, harness, and tools

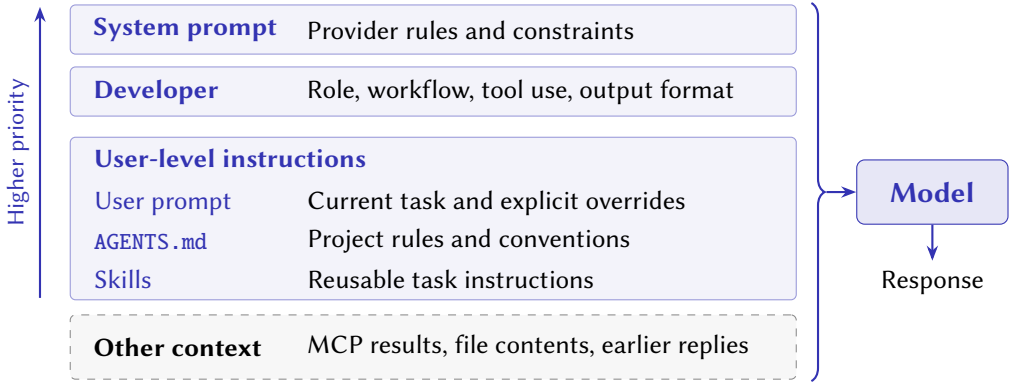
The harness coordinates model calls, context, and tool use



The harness returns tool results to the model so it can continue the task

Context and instruction priority

Context: information available for the next model response

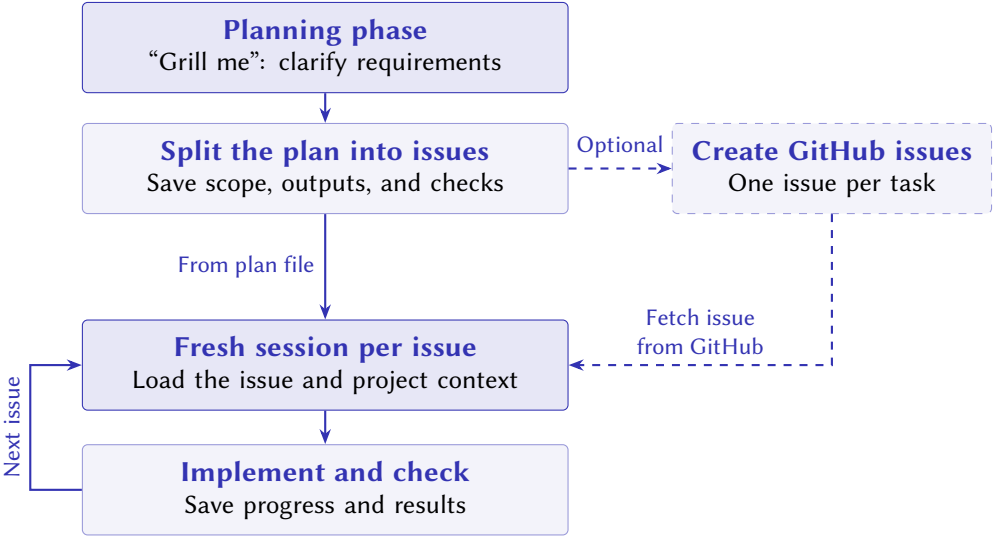


Explicit user requests can override project and skill guidance

Managing context and planning long tasks

- **Goal:** avoid the "dumb zone", where the agent becomes less reliable as context grows (Matt Pocock)
- Start a new session for independent tasks
- Partition long tasks into stages
  1. Before implementation, use "grill me" to have the agent clarify the goal, assumptions, and requirements through questions
  2. Save the stages, outputs, and checks in a plan file
  3. Implement each stage in a fresh context
- Optional: turn the plan into GitHub issues, one issue per task
  - Start a new agent session for each issue
  - Agents can create and read issues automatically through the GitHub CLI (`gh`) or a connected GitHub MCP server

Planning and executing long tasks



Understanding variable results

- The same prompt will produce different code each time
  - Results depend on context, including existing code in the project
  - LLMs are stochastic: even if everything is identical, results will generally differ
- Solutions may differ in complexity and programming constructs
  - For example, a loop in one response and a vectorized function in another
- Results vary by programming language
  - Widely used languages, such as Python, have more public code examples
  - LLMs may produce poorer suggestions for less common languages

- Implication: use planning mode to decide on important aspects of your program before implementation

Fighting LLM bloat (a.k.a. the “wall of text”)

- LLMs tend to produce endless text/code that is becoming incomprehensible to humans (worse since GPT Astra)
- You need to specifically instruct LLMs to simplify:
  - Request well-structured Markdown and brief bullet points for plans
  - Reject useless unit tests (they love unit tests)
  - Reject unnecessary error checking and parameter validation for internal code
  - Ask for modular code: shorter functions are easier to understand
  - Ask for line comments explaining complex operations

Legal issues

- LLMs may infringe on copyright: the training data may be subject to license restrictions

Agentic AI models do not run on your computer

- Everything you enter and all your code files will be uploaded to servers operated by Microsoft/Google/OpenAI/Anthropic/Amazon/etc.
- You may not have permission to share your code or data with third parties (e.g., customer data)
- Check data retention policies (ideally: zero retention)
- Do not allow your prompts/data to be used for training (not possible with free plans)

Customizing your agent

Agent instructions: project and personal rules

- Store instructions that apply across tasks
- Plain Markdown files with project rules or personal preferences
  - Python environment, code style, commands, or files to leave unchanged
- Loaded at the start of a session; keep rules short and specific
- Project instructions: `AGENTS.md` in the project's root folder
- Personal instructions:

Agent	Personal file
Claude	`~/.claude/CLAUDE.md`
Codex	`~/.codex/AGENTS.md`
OpenCode	`~/.config/opencode/AGENTS.md`
Gemini	`~/.gemini/GEMINI.md`

~ is the home folder: `/Users/alice` on macOS, `C:\Users\alice` on Windows

Claude instructions | Codex instructions | OpenCode instructions | Gemini instructions

Example: an AGENTS.md file

File: AGENTS.md

```
# Agent Guidelines for TECH2-H26

- This file contains project-specific instructions for AI agents
  working in this repository

## Python Environment

- The Python code in this repository is intended to be run with
  the ‘TECH2‘ conda environment, which is already present on the
  system
- Run Python and Ruff through this environment, for example with
  ‘conda run -n TECH2 ruff ...‘
```

Skills: reusable instructions for repeated tasks

- Create skills for repeated tasks
- A skill is a folder with task instructions in `SKILL.md`
  - It can also contain scripts, examples, or templates
- Loaded on demand: name and description first; full instructions when needed
- Where to save skills: One subfolder per skill, containing `SKILL.md`

Agent	Project folder	Personal folder
Claude	`.claude/skills/`	`~/.claude/skills/`
OpenCode	`.opencode/skills/`	`~/.config/opencode/skills/`
Gemini	`.gemini/skills/`	`~/.gemini/skills/`
Other	`.agents/skills/`	`~/.agents/skills/`

Claude skills | Codex skills | OpenCode skills | Gemini skills

Example: a SKILL.md file

File: .agents/skills/code-style/SKILL.md

```
---
name: code-style
description: Fix formatting, linting issues, comments, and docstrings in
  user-specified Python files.
disable-model-invocation: true
---

# Python Code Style

- Before the manual review, start by running ‘ruff format‘ on the
  specified paths, using the project’s existing Python environment
  and Ruff configuration
```

Your turn

Create a skill named `inspect-csv` with the following behavior:

1. The user specifies the path to a CSV file
2. The agent identifies and reports the separator (e.g., comma, semicolon, or tab)
3. The agent lists all variables (column names) in a table

Store the skill in the project-specific skills directory for your agent.

Test the skill on `data/CPI.csv` and `data/SCE-Apr-2014.csv`.

Hands-on session

Solving last year's term paper with AI

- Clone the repository and open its folder in your agent:
  https://github.com/richardfoltyn/TECH2-H26-ai-week

Repository layout

- `TASKS.md`: term-paper tasks
- `data/`: survey and CPI data
- `AGENTS.md`: project instructions for the agent
- `.claude/skills/` and `.agents/skills/`: reusable skills

Solving `TASKS.md` one part at a time

1. Discuss the approach with the agent
2. Implement the solution in a notebook
3. Run the notebook and fix any errors
4. Commit the completed part before moving to the next

Cheat sheets for CLI agents

Claude Code: CLI cheat sheet

`@`*path*
Add a file's contents or a directory listing; `@` opens suggestions

`/model`
Choose the model

`/effort high`
Set thinking effort; available levels depend on the model

`/clear`
Start a fresh conversation

`/compact`
Summarize the conversation to free up context

`/usage`
Show session cost, plan limits, and activity

`/status`
Show model, account, and connectivity

`/statusline`
Describe what the footer should display

/skill-name task
Invoke an installed skill by name

`/plan` *task*
Plan a change before implementing it

`/permissions`
Manage which actions require approval

`/resume`
Return to an earlier conversation

`/diff`
Inspect file changes

`/btw` *question*
Ask a side question outside the main conversation

Official command reference | Commands depend on version and account; type / for the menu

Codex: CLI cheat sheet

`@`*path*
Search files and insert a path reference

`$`*skill-name task*
Invoke an installed skill by name

`/model`
Choose model and supported thinking effort

`/new`
Start a new chat with fresh context

`/clear`
Start a new chat and clear the terminal view

`/compact`
Summarize the chat to free up context

`/status`
Check model, permissions, and token usage

`/statusline`
Choose and reorder footer fields

`/skills`
Browse and select available skills

`/plan` *task*
Switch to planning a task

`/permissions`
Adjust approval and access settings

`/resume`
Continue an earlier chat

`/review`
Review changes for bugs; `/diff` shows the file changes

`/btw` *question*
Open a separate side chat; alias: `/side`

Official command reference | Skill invocation

Antigravity CLI (agy): cheat sheet

`@`*path*
Select a file or folder to reference

`/model`
Choose the model

`/effort high`
Set thinking effort; supported levels depend on the model

`/clear`
Reset conversation context; alias: `/new`

`/context`
Inspect what occupies the context window

`/usage`
Show model quota usage

`/credits`
Show AI credit balance and purchase links

`/statusline`
Toggle the footer; pass a script path for a custom display

/skill-name task
Invoke an installed skill by name

`/skills`
Browse skills; `/skills reload` refreshes the list

`/plan` *task*
Draft an implementation plan

`/resume`
Reopen an earlier conversation

`/diff`
Inspect file changes, turns, and commits

`/btw` *question*
Ask a side question while the task runs

Official command reference | Skill invocation

OpenCode: CLI cheat sheet

`@`*path*
Search and select a file to attach its contents to the prompt

`Ctrl+P`
Open the action menu; Open settings changes sidebar, thinking display, and more

`/models`
Choose the model

`/effort`
Select a model variant, such as thinking effort; `Ctrl+T` cycles variants

`/new`
Start a fresh session; alias: `/clear`

`/usage`
View usage limits from supported providers

Official TUI reference

`/skills`
Select a skill to attach to the prompt

`/agents`
Switch agents, e.g., Build or Plan; `Shift+Tab` cycles agents

`/sessions`
Return to an earlier session

`/diff`
Open the file-change viewer

`/undo`
Revert the latest message and its file changes; `/redo` restores them

`/btw` *question*
Ask a side question without adding it to conversation context

GitHub Copilot CLI: cheat sheet

`/model`
Choose model and reasoning effort

`/clear`
Start a fresh conversation; alias: `/new`

`/compact`
Summarize the chat to free up context

`/context`
Inspect context window usage

`/usage`
Show session usage and token totals

`/statusline`
Choose what appears in the footer

`/skills`
Browse and manage available skills

`/skill-name task`
Invoke an installed skill by name

`/plan` *task*
Plan a change before coding

`/permissions`
Manage approval modes; `/permissions reset` clears session approvals

`/resume`
Continue an earlier session

`/review`
Review code changes

`/diff`
Inspect file changes (experimental)

`/btw` *question*
Ask a side question without adding it to conversation history

Official command reference | Skill invocation

Additional resources

Video tutorials: Claude Code for economists

Paul Goldsmith-Pinkham, with Markus Brunnermeier
Eight episodes from Markus' Academy

1. **Getting Started**: https://youtu.be/HzgByl5ZsWE
2. **Data Analysis**: https://youtu.be/Rp17XUPxa4I
3. **Web Scraping**: https://youtu.be/wqLZrKdevHs
4. **Large Datasets**: https://youtu.be/4uwI1-9DafU
5. **Writing and Thinking**: https://youtu.be/BxfSiB3Moyo
6. **Skills**: https://youtu.be/a03ehomPqMA
7. **Permissions and OpenClaw**: https://youtu.be/8Jnx5rL_Gfk
8. **Integration and Collaboration**: https://youtu.be/EcloxLPcRsY